Reddit Information Diffusion and Social Network Analysis

Comprehensive Technical Manual

Senior Data Scientist & Software Architect

April 27, 2026

Contents

1 Executive Summary **2**

1.1 High-Level Architecture 2

1.1.1 System Topology 2

2 Technical Theory and Mathematical Models **4**

2.1 Graph Theory Fundamentals 4

2.2 Sentiment Analysis and Signed Graphs 4

2.3 Node2Vec Embeddings 4

2.4 Uniform Manifold Approximation and Projection (UMAP) 5

2.5 Burt's Constraint Index and Structural Holes 5

2.6 Independent Cascade Model (ICM) 5

3 Data Path Walkthrough **7**

4 Component Glossary **8**

5 Line-by-Line Code Analysis **9**

5.1 `main.py`: The Master Pipeline Controller 9

5.2 `local_collector.py`: High-Performance Offline Archiver 11

5.3 `global_analyzer.py`: Macro-Scale Community Synthesis 12

5.4 `processing_engine.py`: Data Science Preprocessing 13

5.5 `latent_modeler.py`: High-Dimensional Embeddings 14

5.6 `advanced_analytics.py` & `diffusion_simulator.py` 14

6 Detailed Trace Analysis: Processing a Single Information Cascade **16**

6.1 The Hypothetical Scenario 16

6.2 Phase 1: Ingestion and Decompression 16

6.3 Phase 2: Sentiment Enrichment 17

6.4 Phase 3: Pruning the Fluff 17

6.5 Phase 4: Latent Mapping 17

6.6 Phase 5: Brokerage Analytics 18

6.7 Phase 6: The Diffusion Contagion 18

7 Conclusions and Scalability Directives **19**

Chapter 1

Executive Summary

1.1 High-Level Architecture

This technical manual outlines the complete architecture, theoretical foundations, and line-by-line implementation details of the **JIM-JAM** Reddit Social Network Analysis (SNA) and Information Diffusion platform. The platform is designed to ingest massive streams of social interaction data, extract underlying latent behavioral patterns, and model the stochastic spread of information across complex topologies.

The architecture is built upon a modular, highly scalable pipeline capable of both live streaming via the Reddit API (PRAW) and high-throughput offline batch processing using ZStandard compressed NDJson archives.

1.1.1 System Topology

The global topology of the system consists of the following core stages:

1. **Data Acquisition Layer**: Comprising `RedditCollector` and `LocalDumpCollector`, responsible for flattening hierarchical comment trees into directed bipartite graphs.

2. **Enrichment Layer**: Utilizing the `SentimentEngine` to compute valence scores, transitioning the standard topological graph into a Signed Graph.

3. **Processing and Pruning**: The `AntiFluffEngine` acts as a stochastic filter, applying quantile-based edge removal and feature scaling via Z-score normalization.

4. **Latent Modeling**: Employing Node2Vec for random-walk based high-dimensional embeddings and UMAP for non-linear dimensionality reduction to identify behavioral echo chambers.

5. **Topological Analytics**: The `AdvancedAnalytics` and `GlobalRedditAnalyzer` modules calculate Burt's Constraint Index, identify Network Motifs, and compute cross-community Jaccard similarities.

6. **Simulation Engine**: The `DiffusionSimulator` models information cascades via the Independent Cascade Model (ICM).

Chapter 2

Technical Theory and Mathematical Models

2.1 Graph Theory Fundamentals

A Reddit conversation thread is modeled as a directed multigraph $G = (V, E, W)$, where V represents the set of authors (nodes), E represents the set of reply interactions (edges), and W represents the aggregation of interaction frequencies (edge weights). Let $e_{ij} \in E$ denote a directed edge from node i to node j, indicating that user i replied to user j.

2.2 Sentiment Analysis and Signed Graphs

The `SentimentEngine` employs the VADER (Valence Aware Dictionary and sEntiment Reasoner) model to assign a compound sentiment score $s_{ij} \in [-1, 1]$ to each interaction. This transforms G into a Signed Graph $G^{\pm}$, where edges carry emotional polarity. The compound score c is computed by summing the valence scores of each word v_k in the lexicon, adjusted according to syntactic rules, and normalized:

$$c = \frac{\sum_k v_k}{\sqrt{(\sum_k v_k)^2 + \alpha}} \tag{2.1}$$

where α is a normalization constant (typically 15).

2.3 Node2Vec Embeddings

The `LatentModeler` leverages the Node2Vec algorithm to learn a mapping function $f : V \to \mathbb{R}^d$ from nodes to high-dimensional representations. The objective is to maximize the log-probability of observing a network neighborhood $N_S(u)$ for a node u conditioned on its feature representation, using a biased random walk strategy. The objective function

is given by:

$$\max_f \sum_{u \in V} \log \Pr(N_S(u) \mid f(u)) \tag{2.2}$$

The random walk transitions from node v to x, having just traversed edge (t, v), are governed by the unnormalized transition probability $\pi_{vx} = \alpha_{pq}(t, x) \cdot w_{vx}$, where w_{vx} is the edge weight and the search bias α_{pq} is defined as:

$$\alpha_{pq}(t, x) = \begin{cases} \frac{1}{p} & \text{if } d_{tx} = 0 \\ 1 & \text{if } d_{tx} = 1 \\ \frac{1}{q} & \text{if } d_{tx} = 2 \end{cases} \tag{2.3}$$

where p is the return parameter, q is the in-out parameter, and d_{tx} is the shortest path distance.

2.4 Uniform Manifold Approximation and Projection (UMAP)

Following the extraction of high-dimensional embeddings, UMAP is utilized for manifold learning. UMAP constructs a fuzzy simplicial complex representation of the data and optimizes a low-dimensional layout to minimize the cross-entropy C between the high-dimensional fuzzy set edge weights v_{ij} and the low-dimensional weights w_{ij}:

$$C = \sum_{i \neq j} \left(v_{ij} \log \left(\frac{v_{ij}}{w_{ij}} \right) + (1 - v_{ij}) \log \left(\frac{1 - v_{ij}}{1 - w_{ij}} \right) \right) \tag{2.4}$$

2.5 Burt's Constraint Index and Structural Holes

In the `AdvancedAnalytics` module, brokerage power is quantified using Ronald Burt's Constraint Index. The constraint c_i on node i measures the extent to which i's connections are to nodes that are connected to one another:

$$c_i = \sum_j \left(p_{ij} + \sum_{q \neq i,j} p_{iq} p_{qj} \right)^2 \tag{2.5}$$

where p_{ij} is the proportional strength of i's relationship with j. Nodes with minimal constraint are identified as "Shadow Brokers" bridging isolated echo chambers.

2.6 Independent Cascade Model (ICM)

The `DiffusionSimulator` implements the Independent Cascade Model. In ICM, time unfolds in discrete steps $t = 0, 1, 2, \ldots$. When node u first becomes active at time t, it

is given a single chance to activate each currently inactive neighbor v with a success probability $p_{u,v}$. The total expected spread from a seed set S, denoted $\sigma(S)$, is given by the expectation over all possible cascade realizations:

$$\sigma(S) = \mathbb{E}_X[|A(X, S)|] \tag{2.6}$$

where X is a specific instantiation of edge activations.

Chapter 3

Data Path Walkthrough

The signal data flows through the application via a highly structured pipeline, analogous to an optical transmission system traversing a series of amplifiers and filters:

1. **Ingestion (Transponder)**: Data originates either from live API requests via `RedditCollector` or from raw offline `.zst` archives through `LocalDumpCollector`. In offline mode, the data is decompressed via a streaming ZStandard buffer to minimize RAM footprint.

2. **Enrichment (Modulation)**: The raw textual payloads are pushed through the `SentimentEngine`, mapping textual sentiment to a numerical continuous spectrum (-1.0 to 1.0).

3. **Pruning (Dispersion Compensation)**: `AntiFluffEngine` drops the lowest quartile (or arbitrary q-th percentile) of edge weights. This acts as a low-pass filter against isolated, meaningless interactions, concentrating the graph signal-to-noise ratio.

4. **Mapping (Spectral Analysis)**: `LatentModeler` extracts the Node2Vec topological representations and flattens them using UMAP into a 2D Euclidean space for clustering.

5. **Analysis (Demultiplexing)**: The `AdvancedAnalytics` isolates target broker nodes by computing structural constraint limits.

6. **Simulation (Signal Propagation)**: The extracted target brokers are injected as seeds into the `DiffusionSimulator` to model maximal network activation spread.

Chapter 4

Component Glossary

- `LocalDumpCollector`: High-throughput offline archiver utilizing `zstandard` streaming to parse NDJson arrays into Pandas DataFrames.
- `RedditCollector`: Live REST API hook into PRAW to extract directed acyclic comment trees.
- `SentimentEngine`: NLP wrapper integrating VADER sentiment heuristics to evaluate text payload emotional valence.
- `AntiFluffEngine`: Topological data pre-processor designed for weight quantile trimming and Z-score degree scaling.
- `LatentModeler`: Machine Learning wrapper orchestrating `node2vec` vectorizations and `umap` projections.
- `AdvancedAnalytics`: Graph theory unit executing Burt's constraint evaluations and Triadic Motifs aggregations.
- `DiffusionSimulator`: Epidemic modeling unit utilizing the stochastic ICM paradigm to map info-cascades.
- `GlobalRedditAnalyzer`: Macro-scale community analyzer mapping cross-subreddit Jaccard similarities and user versatility metrics.

Chapter 5

Line-by-Line Code Analysis

5.1 main.py: The Master Pipeline Controller

The main.py script orchestrates the entire workflow, acting as the primary entrypoint for the system's execution pipeline.

```
import sys
import pandas as pd
from reddit_collector import RedditCollector
from local_collector import LocalDumpCollector
from torrent_downloader import TorrentManager
from config import TARGET_SUBREDDITS
import os
from processing_engine import AntiFluffEngine
from sentiment_engine import SentimentEngine
from latent_modeler import LatentModeler
from advanced_analytics import AdvancedAnalytics
from diffusion_simulator import DiffusionSimulator
from visualizer import SocialVisualizer
from global_analyzer import GlobalRedditAnalyzer
```

Listing 5.1: Main Execution Pipeline - Import and setup

Analysis: The initial segment imports all required standard and local modules. Crucially, the local imports represent the distinct phases of the data processing pipeline, analogous to modular hardware components being wired together.

```
def run_pipeline(subreddit_name, praw_creds, mode='auto'):
    print(f"=== Starting Reddit SNA Pipeline ===")

    # 1. Collection (Automatic Source Selection)
    dump_dir = "data/reddit/subreddits25"
    dump_path = os.path.join(dump_dir, f"{subreddit_name}_comments.zst"
   )
```

Listing 5.2: run_pipeline Function Setup

Analysis: The run_pipeline function is the core execution loop. It takes a target subreddit, API credentials, and an operational mode. It first constructs the expected path

for local compressed archives.

```
    if os.path.exists(dump_path) and (mode == 'auto' or mode == 'archive'):
        # ARCHIVE MODE: Fast processing of local dumps
        print(f"[*] Found local archival data for r/{subreddit_name}. Switching to ARCHIVE mode.")
        collector = LocalDumpCollector(dump_dir=dump_dir)
        df_raw = collector.stream_subreddit(subreddit_name, limit=20000)
    elif mode == 'live':
        print(f"[*] Connecting to Reddit API (LIVE mode)...")
        collector = RedditCollector(
            client_id=praw_creds['client_id'],
            client_secret=praw_creds['client_secret'],
            user_agent=praw_creds['user_agent']
        )
        df_raw = collector.fetch_subreddit_interactions(subreddit_name, limit=50)
```

Listing 5.3: Data Acquisition Switch Logic

Analysis: The logic gate here defines the acquisition pathway. If local ZStandard archives are present, it defaults to the high-throughput `LocalDumpCollector`, parsing up to 20,000 interactions sequentially. If invoked in `live` mode, it dynamically instantiates the `RedditCollector` using PRAW credentials to pull fresh threads from the Reddit REST API.

```
    # 2. Sentiment Enrichment (Anti-Fluff Feature)
    print("[*] Enriching interactions with sentiment...")
    if 'text' not in df_raw.columns:
        df_raw['text'] = "Mock comment text for r/" + subreddit_name

    sentiment_engine = SentimentEngine()
    df_enriched = sentiment_engine.enrich_interactions(df_raw, {})
```

Listing 5.4: Enrichment and Sentiment Injection

Analysis: Here, the textual payloads of interactions are enriched via the `SentimentEngine`. A fallback mock text is applied if the column is missing to prevent pipeline crashes. The `enrich_interactions` method computes continuous valence scores for each row.

```
    # 3. Processing (Quantile Pruning)
    engine = AntiFluffEngine(df_enriched)
    G = engine.prune_edges(q=0.5) # Focus on top 50% signal

    # 4. Latent Modeling
    modeler = LatentModeler(G)
    embeddings = modeler.generate_embeddings(dimensions=16, walk_length=10, num_walks=50)
    projection = modeler.project_to_2d()
    SocialVisualizer.plot_umap(projection)
```

Listing 5.5: Topological Pruning and Latent Extraction

Analysis: The raw DataFrame is converted into a NetworkX `DiGraph` by the `AntiFluffEngine`. The call to `prune_edges(q=0.5)` structurally removes edges failing to meet the median weight threshold, enhancing signal clarity. Subsequently, the `LatentModeler` calculates 16-dimensional Node2Vec embeddings using 50 stochastic random walks of length 10 per node. These embeddings are reduced to 2D via UMAP and visually dumped by `SocialVisualizer`.

```
    # 5. Advanced Analytics (Brokers & Motifs)
    analytics = AdvancedAnalytics(G)
    brokers = analytics.calculate_structural_holes()

    # 6. Diffusion Simulation
    seeds = brokers.index[:2].tolist() if not brokers.empty else []
    if seeds:
        simulator = DiffusionSimulator(G)
        active, history = simulator.run_icm(seeds, p=0.2)
        SocialVisualizer.plot_diffusion_curve(history)
```

Listing 5.6: Broker Identification and Cascade Simulation

Analysis: Finally, `AdvancedAnalytics` computes Burt's Constraint Index. The top two nodes with the lowest constraint (highest broker power) are injected as initial active seeds into the `DiffusionSimulator`. The ICM is executed with a propagation probability of $p = 0.2$ across adjacent edges, and the cumulative activation history is serialized to an output visual.

5.2 `local_collector.py`: High-Performance Offline Archiver

The `local_collector.py` script is engineered to bypass memory bottlenecks when dealing with multi-gigabyte JSON dumps.

```
class LocalDumpCollector:
    def stream_subreddit(self, subreddit_name, limit=10000):
        # ... filename parsing ...
        dctx = zstd.ZstdDecompressor()

        with open(filepath, 'rb') as fh:
            with dctx.stream_reader(fh) as reader:
                import io
                text_stream = io.TextIOWrapper(reader, encoding='utf-8'
    )

                for line in text_stream:
                    try:
                        comment = json.loads(line)
                        parent_id = comment.get('parent_id', '')
                        # Append interaction details to list
                    except Exception as e:
```

```
                        continue
```

Listing 5.7: Local Dump Processing with ZStandard

Analysis: Rather than using `pd.read_json()` which loads the entire JSON into RAM, this module utilizes the `zstd.stream_reader`. It decodes the binary stream using `io.TextIOWrapper` and applies `json.loads()` iteratively. This effectively restricts time complexity to $O(N)$ and space complexity to $O(K)$ where K is the processing `limit`.

```
        # 1. Map parent_id to actual author if available in the same batch
        df['target'] = df['target_id'].apply(lambda x: x.split('_')[-1] if '_' in str(x) else x)

        id_to_author = dict(zip(df['comment_id'], df['source']))
        df['target_author'] = df['target'].map(id_to_author)
        df['target'] = df['target_author'].fillna(df['target'])
```

Listing 5.8: Resolving Parent IDs to Author Names

Analysis: In Reddit datasets, replies link to a `parent_id` rather than the target author. This logic constructs an in-memory `id_to_author` hash map from the current processing chunk. It attempts an $O(1)$ lookup for the `target_id`. If the target comment exists outside the current window chunk, the system elegantly falls back to using the raw `id` string via `fillna`.

5.3 `global_analyzer.py`: Macro-Scale Community Synthesis

The `GlobalRedditAnalyzer` shifts perspective from intra-subreddit micro-dynamics to inter-subreddit macro-topologies.

```
    def calculate_jaccard_similarity(self):
        subreddits = sorted(list(self.subreddit_sentiment.keys()))
        n = len(subreddits)
        matrix = np.zeros((n, n))

        # Pre-calculate user sets for each subreddit
        sub_users = {sub: set() for sub in subreddits}
        for user, subs in self.user_subreddit_map.items():
            for sub in subs:
                if sub in sub_users:
                    sub_users[sub].add(user)

        for i in range(n):
            for j in range(i, n):
                sub1, sub2 = subreddits[i], subreddits[j]
                set1, set2 = sub_users[sub1], sub_users[sub2]

                intersection = len(set1.intersection(set2))
```

```
                union = len(set1.union(set2))

                similarity = intersection / union if union > 0 else 0
                matrix[i, j] = similarity
                matrix[j, i] = similarity

        return pd.DataFrame(matrix, index=subreddits, columns=
    subreddits)
```

Listing 5.9: Computing the Jaccard Similarity Matrix

Analysis: The mathematical foundation here is the Jaccard index: $J(A, B) = \frac{|A \cap B|}{|A \cup B|}$. The code first constructs inverse mappings, translating `user_subreddit_map` into distinct disjoint sets per subreddit. It iterates over the upper triangular matrix (via j in range i to n), computing intersection operations via highly-optimized Python `set` primitives. The calculated scalar populates the symmetrical indices (i, j) and (j, i) inside a NumPy tensor before projection to a Pandas DataFrame.

5.4 `processing_engine.py`: Data Science Preprocessing

The `AntiFluffEngine` transforms and normalizes topological properties prior to modeling.

```
    def scale_node_features(self):
        degrees = dict(self.G.degree())
        betweenness = nx.betweenness_centrality(self.G)

        features_df = pd.DataFrame({
            'degree': list(degrees.values()),
            'betweenness': list(betweenness.values())
        }, index=list(degrees.keys()))

        scaler = StandardScaler()
        scaled_features = scaler.fit_transform(features_df)

        return pd.DataFrame(scaled_features, columns=features_df.
    columns, index=features_df.index)
```

Listing 5.10: Z-Score Normalization of Centrality Metrics

Analysis: Topological metrics like degree centrality adhere strictly to power-law distributions, presenting extreme long tails that heavily skew downstream machine learning clustering algorithms. This logic calculates raw degree sequences and Betweenness Centrality limits. It relies upon `sklearn.preprocessing.StandardScaler` to execute standard score transformations: $z = \frac{(x-\mu)}{\sigma}$. The returned DataFrame ensures topological features exert uniform inductive bias.

5.5 latent_modeler.py: High-Dimensional Embeddings

The LatentModeler leverages representation learning to identify covert network behaviors.

```
    def generate_embeddings(self, dimensions=64, walk_length=30, num_walks=100):
        n2v = Node2Vec(self.G, dimensions=dimensions, walk_length=walk_length, num_walks=num_walks)
        self.model = n2v.fit(window=10, min_count=1, batch_words=4)
        self.embeddings = {node: self.model.wv[node] for node in self.G.nodes()}
        return self.embeddings

    def project_to_2d(self):
        nodes = list(self.embeddings.keys())
        vectors = np.array([self.embeddings[node] for node in nodes])
        reducer = umap.UMAP(n_neighbors=15, min_dist=0.1, metric='cosine')
        u_map = reducer.fit_transform(vectors)
        return pd.DataFrame(u_map, columns=['UMAP_1', 'UMAP_2'], index=nodes)
```

Listing 5.11: Executing Random Walks and UMAP

Analysis: The generate_embeddings method instantiates a Node2Vec framework computing probability gradients via Skip-Gram architecture. The contextual window size is statically bound to 10. The project_to_2d function collapses the resultant high-dimensional matrices using UMAP. Crucially, the non-metric cosine distance evaluates angular similarity, completely disregarding vector magnitude differences—an optimal technique for evaluating abstract latent structures over purely volume-based similarities.

5.6 advanced_analytics.py & diffusion_simulator.py

```
class AdvancedAnalytics:
    def calculate_structural_holes(self):
        constraint = nx.constraint(self.G)
        df = pd.DataFrame.from_dict(constraint, orient='index', columns=['constraint'])
        return df.sort_values('constraint', ascending=True)
```

Listing 5.12: Burt's Constraint Index Computation

Analysis: The script delegates the computationally intensive $O(V \cdot K^2)$ computation (where K is average degree) of local clustering and closure coefficients to the optimized networkx C-bindings. Outputting ascending sorted values deliberately isolates nodes with absolute minimal structural constraints, exposing them as network bridges.

```
class DiffusionSimulator:
    def run_icm(self, seed_nodes, p=0.1):
        active = set(seed_nodes)
```

```
        newly_active = list(seed_nodes)
        history = [len(active)]

        while newly_active:
            next_active = []
            for node in newly_active:
                for neighbor in self.G.neighbors(node):
                    if neighbor not in active:
                        if random.random() < p:
                            active.add(neighbor)
                            next_active.append(neighbor)
            newly_active = next_active
            history.append(len(active))
        return active, history
```

Listing 5.13: The Independent Cascade Simulation Loop

Analysis: This nested iterative loop simulates a synchronous discrete-time Markov cascade. In each epoch, `newly_active` nodes project probabilistic influence against their out-neighborhood set. The pseudorandom number generator (`random.random()`) evaluates activation success dynamically against barrier threshold p. Only successful state transitions append the neighbor to the `next_active` queue. The topological reach monotonically increases, terminating when the cascade extinguishes (empty `newly_active` state limit).

Chapter 6

Detailed Trace Analysis: Processing a Single Information Cascade

6.1 The Hypothetical Scenario

To fully comprehend the intricate processing pipeline, we will trace a specific, hypothetical interaction cascade. Consider a scenario involving three distinct Reddit users: User A, User B, and User C.

User A is highly active in `r/technology` but rarely ventures elsewhere. User B is a prolific commenter in `r/politics`. User C, however, is moderately active in both communities.

The cascade initiates when User A posts a controversial opinion regarding artificial intelligence regulation in `r/technology`. User C replies to this comment, and subsequently links back to it during a discussion with User B in `r/politics`.

We will trace exactly how the system processes this multi-step cascade, from raw NDJson bytes to a simulated viral contagion.

6.2 Phase 1: Ingestion and Decompression

The data begins its journey inside a multi-gigabyte ZStandard compressed archive file named `technology_comments.zst`.

When `run_pipeline()` initializes the `LocalDumpCollector`, it does not load the gigabytes into RAM. Instead, it utilizes `zstd.stream_reader` to pipe byte-streams through an `io.TextIOWrapper`. The system encounters the first interaction as a raw NDJson string:

```
{
    "author": "User_C",
    "body": "This AI regulation will drastically shift market dynamics.
   ",
    "created_utc": 1672531200,
    "id": "j3x9k1",
```

```
    "parent_id": "t1_h8y2p0",
    "subreddit": "technology"
}
```

Listing 6.1: Raw JSON Interaction

The `LocalDumpCollector` maps the `parent_id` ("t1_h8y2p0") back to User A using the `id_to_author` hash map. The raw byte-string is transformed into a structured directed edge: User_C → User_A with a timestamp and a textual payload.

6.3 Phase 2: Sentiment Enrichment

The structured edge is passed to the `SentimentEngine`. The VADER lexicon evaluates the text payload: "This AI regulation will drastically shift market dynamics...".

The words "drastically" and "shift" might carry neutral or slightly negative weight depending on the context modifiers. The engine computes a valence score, for example, $s_{C,A} = -0.15$. The graph edge is now enriched: it is no longer just a structural link, but a *semantic* vector carrying an emotional polarity score of -0.15.

6.4 Phase 3: Pruning the Fluff

Next, the enriched interaction passes to the `AntiFluffEngine`. The engine aggregates all interactions. Suppose User C replied to User A exactly once, giving this edge a raw `weight` of 1.

The engine calculates the $q = 0.5$ (median) quantile threshold for all interactions across `r/technology`. If the median interaction frequency across the subreddit is exactly 1, the edge might just survive the pruning process (or be culled if the threshold is higher). For our example, we assume User C has engaged with User A multiple times, resulting in a weight of 4. The edge survives the low-pass filter and is solidified into the NetworkX `DiGraph`.

6.5 Phase 4: Latent Mapping

With the graph structurally finalized, the `LatentModeler` initializes Node2Vec. The algorithm performs 50 random walks starting from User C. Because User C is bridged to User A (technology) and User B (politics), the random walks frequently bounce between these two otherwise disconnected clusters.

As a result, the Skip-Gram neural network learns that User C's context window contains both technology-focused nodes and politics-focused nodes. When the 16-dimensional vector is generated, User C will be mathematically positioned in the interstitial space between the two massive topological clusters.

When UMAP subsequently flattens these 16 dimensions down to 2D for the `SocialVisualizer`, User C will be plotted visually in the "no-man's land" between the dense `r/technology` echo chamber and the `r/politics` echo chamber.

6.6 Phase 5: Brokerage Analytics

The `AdvancedAnalytics` module now scans the graph using Burt's Constraint Index. User A has a high constraint: almost all of A's friends are also friends with each other within `r/technology`. User B similarly has high constraint within `r/politics`.

User C, however, possesses edges bridging these distinct components. Consequently, C's constraint index c_C drops significantly:

$$c_C = \sum_j \left(p_{Cj} + \sum_q p_{Cq} p_{qj} \right)^2 \approx 0.12 \tag{6.1}$$

A constraint score of 0.12 is exceptionally low. The system flags User C as a primary "Shadow Broker".

6.7 Phase 6: The Diffusion Contagion

Because User C is flagged as a top broker, `main.py` extracts User C and feeds them into the `DiffusionSimulator` as a `seed_node`.

The Independent Cascade Model (ICM) begins at $t = 0$ with $A_{active} = \{\text{User_C}\}$. At $t = 1$, User C attempts to infect its neighbors (User A and User B) with a probability $p = 0.2$. Using the pseudorandom generator, the system evaluates the cascade. Suppose the transmission to User B succeeds. User B is added to A_{active}.

At $t = 2$, User B now attempts to infect all of their highly-connected neighbors in the `r/politics` subreddit. Because the `r/politics` cluster is extremely dense, the infection spreads rapidly like a chain reaction.

What began as a localized interaction (User C replying to User A) has now, via the structural hole bridged by User C, cascaded into a massive contagion event within an entirely different subreddit. The `visualizer.py` outputs the cumulative sum of these activated nodes over time, generating the classic S-curve seen in epidemiological diffusion graphs.

Chapter 7

Conclusions and Scalability Directives

The JIM-JAM repository serves as a meticulously engineered blueprint for large-scale sociodynamic analysis. The architecture inherently isolates IO-bound bottlenecks via local ZStandard compression streamers, concurrently maintaining topological data structures optimized for random walk sampling and stochastic simulations. Future extensions involve integrating temporal decay parameters across the simulation phase to accurately map diminishing relevancy within the information cascades.